

ARQUITETURAS DE COMPUTADORES

TURMA A1

TÓPICO 3 – MEMÓRIA CACHE (AULA 18)

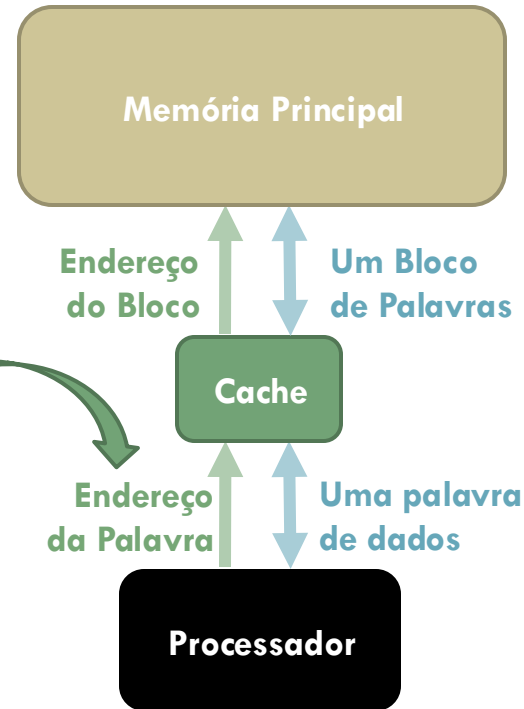
Prof. Vinod Rebello, vinod@ic.uff.br, Sala 413

Acessando Memória Principal

- O processador continua achando que ele está acessando a memória principal, enviando um endereço (com tamanho M bits) da palavra solicitada

M bits do endereço da palavra

- Porém a cache, para explorar localidade espacial, precisa acessar um conjunto de palavras – isto é um bloco.



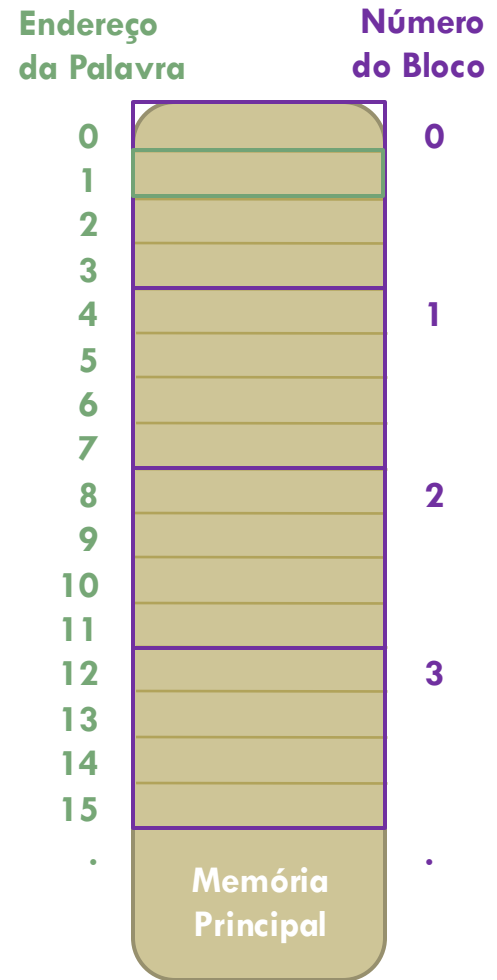
Acessando Memória Principal

- O processador continua achando que ele está acessando a memória principal, enviando um endereço (com tamanho de M bits) da palavra solicitada

M bits do endereço da palavra

- Porém a cache, para explorar localidade espacial, precisa acessar um conjunto de palavras – isto é um bloco.
- Então a MP é dividida em blocos de 2^P palavras de endereços consecutivos

- **Endereço i** fica no **Bloco $(i \div 2^P)$**



Com P = 2

Acessando Memória Principal

- O processador continua achando que ele está acessando a memória principal, enviando um endereço da palavra solicitada

M bits do endereço da palavra

- Porém a cache, para explorar localidade espacial, precisa acessar um conjunto de palavras – um bloco.
- Então a MP é dividida em blocos de 2^P palavras de endereços consecutivos

M - P bits

P bits

Endereço do bloco

Posição da palavra no bloco

Endereço da
Palavra

Endereço do
Bloco

0000
0001
0010
0011
0100
0101
0110
0111
1000
1001
1010
1011
1100
1101
1110
1111
.

00

01

10

11

.



Com $P = 2$

Duas organizações de Cache

- As caches têm uma estrutura diferente de memória principal
 - Além de um bloco de memória, cada entrada ou **linha da cache** precisa guardar alguma informação que identifica qual bloco está sendo armazenado nesta linha
- Existe duas principais organizações (mais uma que é a combinação das duas)
 - Cache associativa
 - Cache com mapeamento direto
 - Cache associativa por conjunto (um híbrido da duas anteriores)
- A diferença entre as duas caches é relacionada à maneira em que como cada uma resolve as seguintes questões:
 - Quando copiamos um bloco de dados da memória principal para a cache, onde exatamente devemos colocá-lo?
 - Como podemos saber se uma palavra já está na cache ou se seu bloco deve ser buscado na memória principal?

Cache Associativa

- Neste tipo de cache, um bloco pode ser carregada em qualquer linha (disponível) da cache
- Cada linha tem três campos: **um bit de validade** que identifica se a linha está guardando um bloco, **o número do bloco** e **o bloco**
 - Custo do cache é relacionado seu tamanho =
 - $L \times (1 + \log_2(\text{Quantidade de blocos na MP}) + \text{Tamanho do bloco})$ bits

L Linhas	Bit de Validade	No. do Bloco	O Bloco			
0						
1						
2						
3						
...						
$L - 1$						

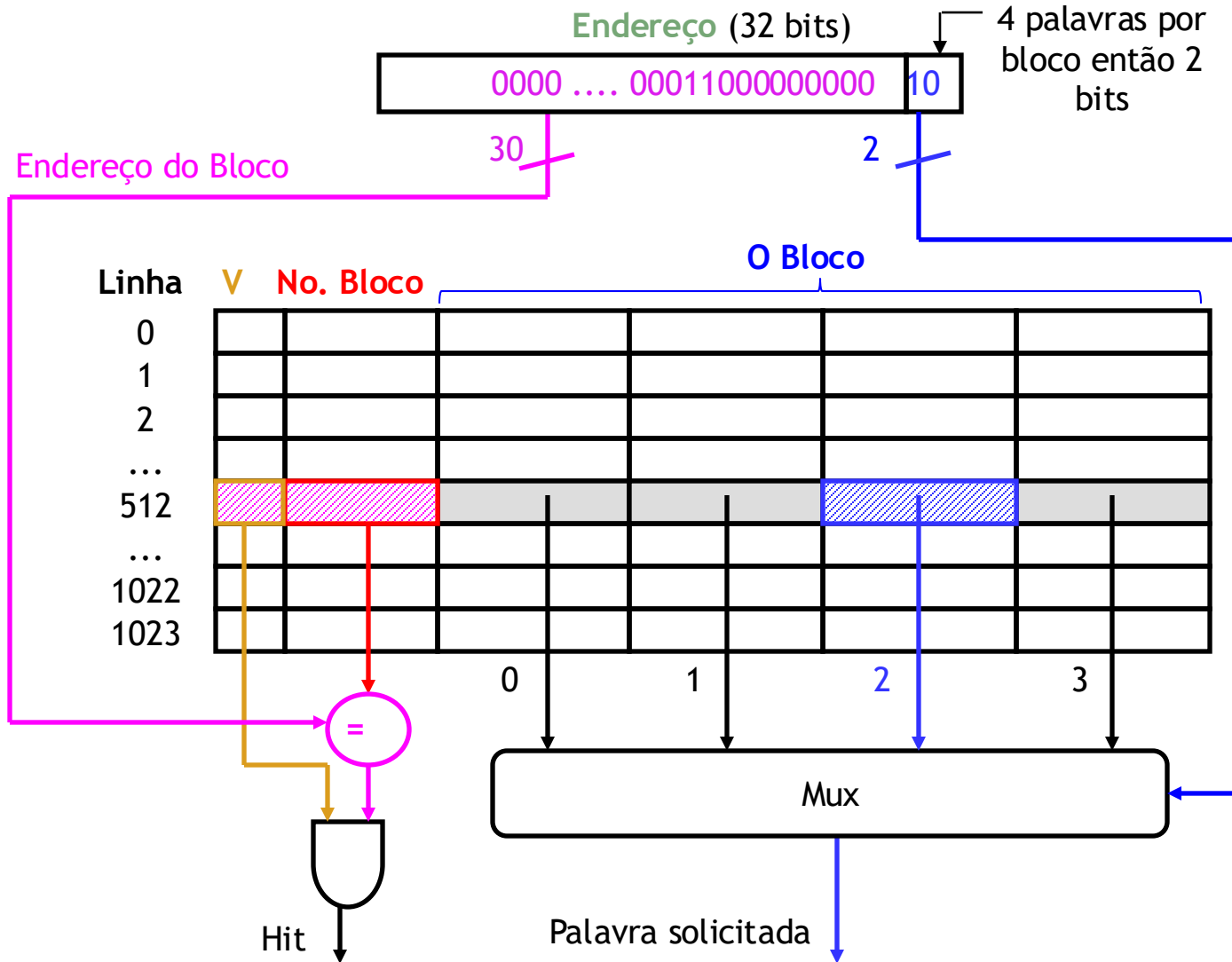
1 bit

$\underbrace{\hspace{10em}}$
 $\log_2(\text{Tam. de MP} \div \text{Tam. do bloco})$

$\underbrace{\hspace{15em}}$
Tam. do bloco = $2^p \times \text{Tamanho da palavra}$

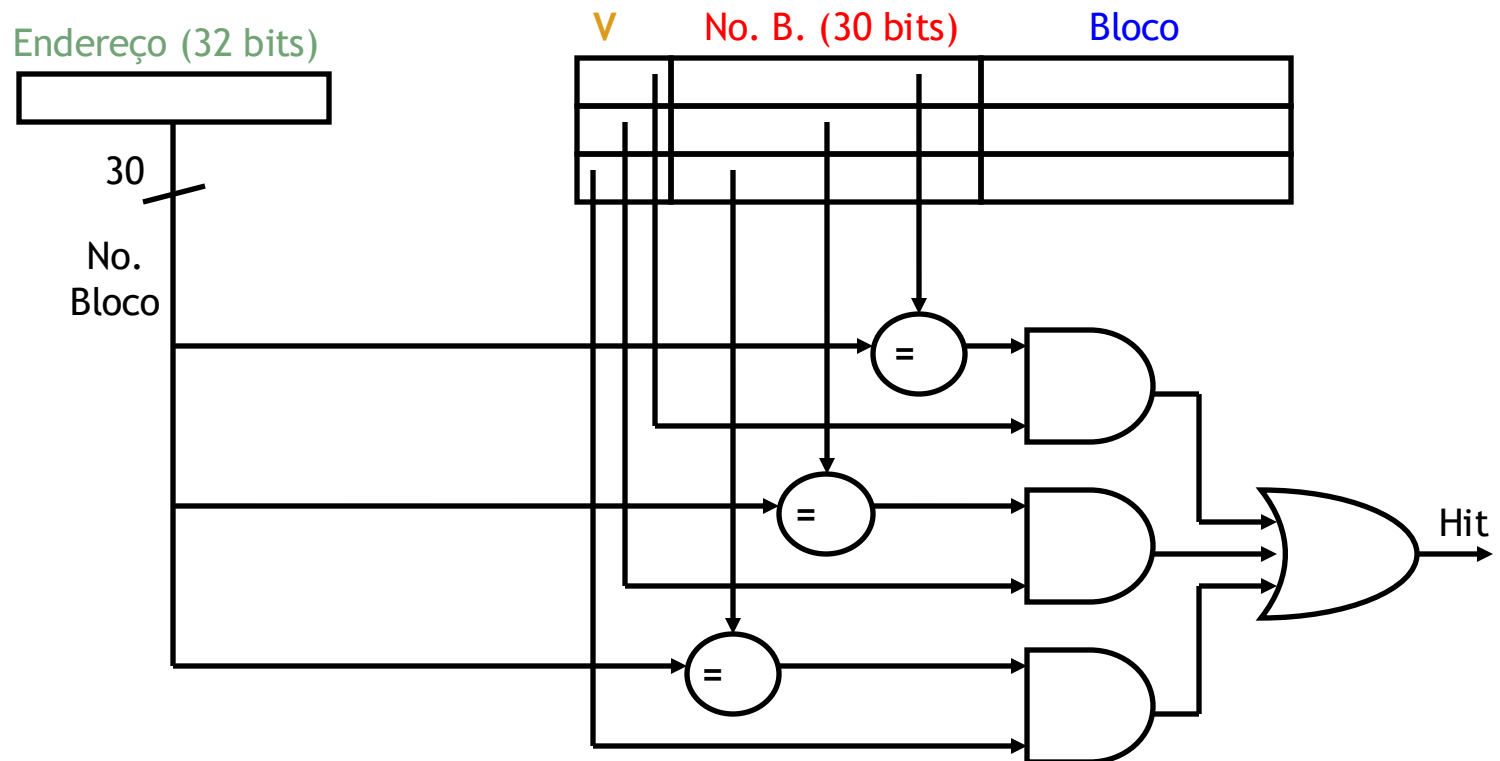
Cache Associativa

- CPU envia um endereço de onde a cache determina o número do bloco que contém a palavra solicitada
- A cache ainda tem que determinar se o bloco está em alguma linha da cache



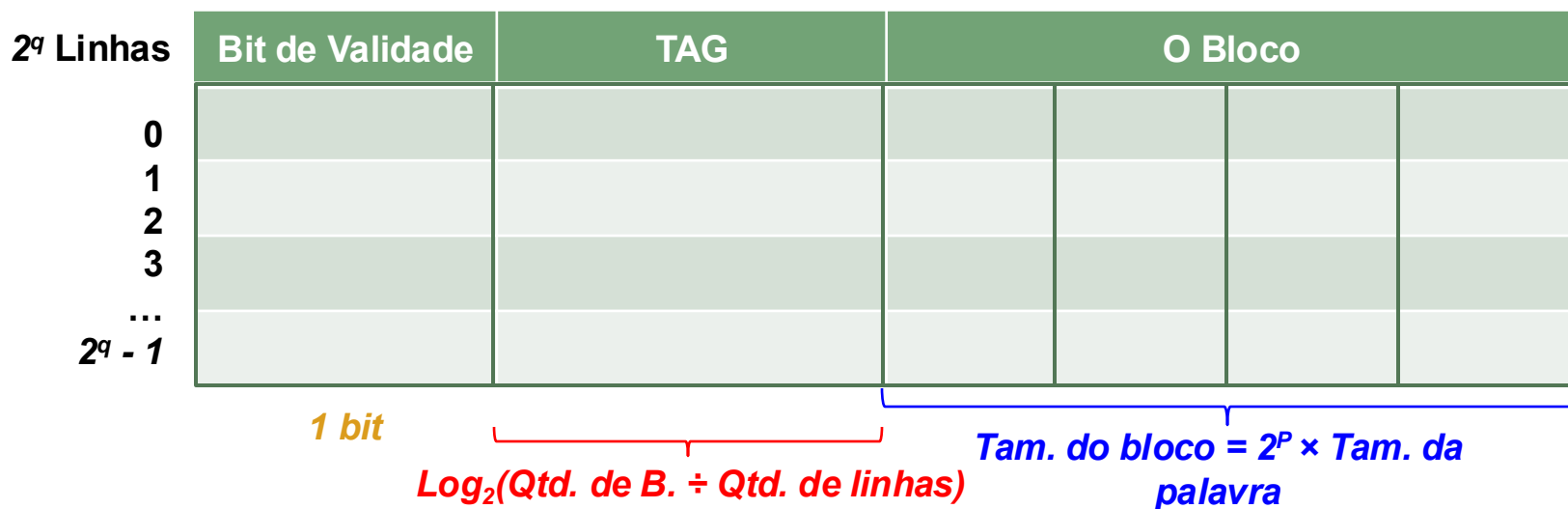
Busca em paralela da Cache Associativa

- Nesta cache um bloco pode ser carregada em qualquer linha de cache
 - Então, para achar um dado bloco, vai precisar buscar no cache inteiro
 - Para não demorar, é feita em paralelo, buscando em todas as linhas no mesmo tempo
 - Isso pode ser bastante caro, dependendo na quantidade de linhas (L) do cache



Cache com Mapeamento Direto

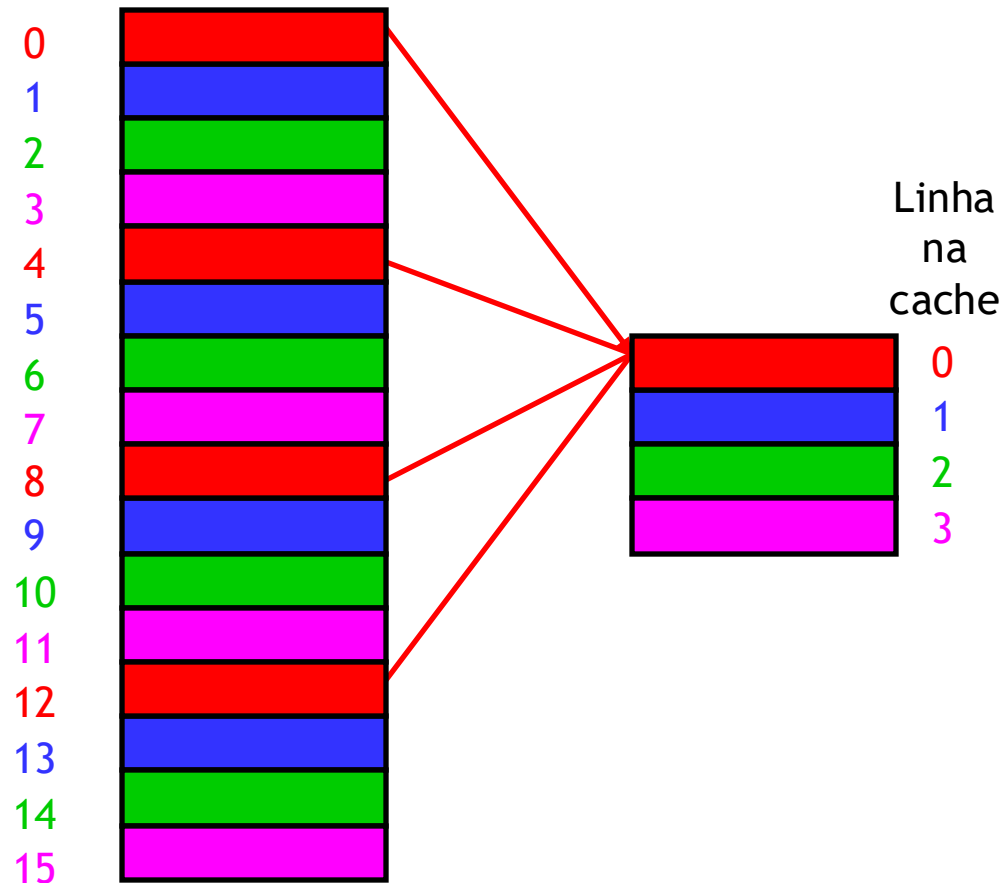
- Neste tipo de cache, cada bloco da memória principal tem **um único lugar (uma linha fixa)** na memória cache
 - Cada linha tem três campos: **o bit de validade** que identifica se a linha está guardando um bloco, **um identificador do bloco (TAG)** e **o bloco**
 - Tamanho da cache = $2^q \times (1 + \text{Log}_2(\text{Qtd de ??}) + \text{Tamanho do bloco})$



Cache com Mapeamento Direto

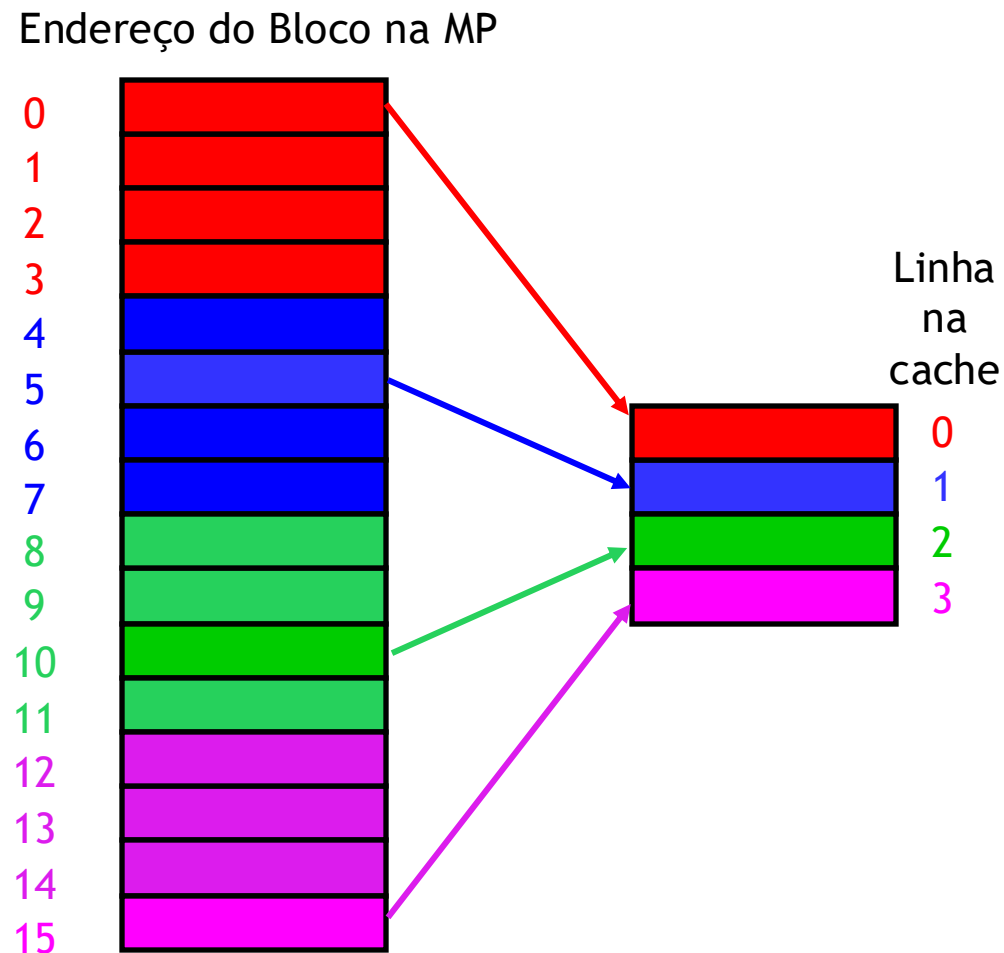
- Cada bloco tem um **único** lugar na memória cache onde pode ser guardado, mas onde?

Endereço do Bloco na MP



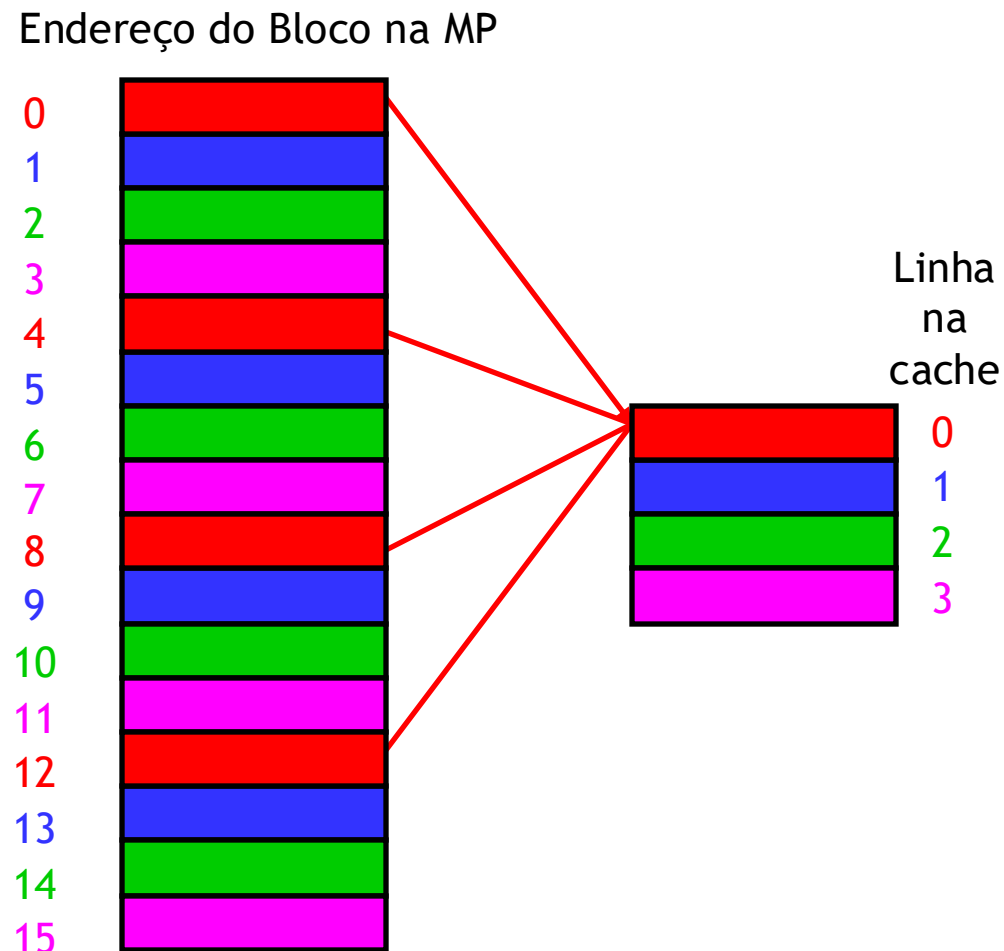
Cache com Mapeamento Direto

- Cada bloco tem um único lugar na memória cache, mas onde?
- Por exemplo, à direita está uma memória principal de 16 blocos e uma cache de 4 linhas.
 - Os locais de memória 0 a 3 são mapeados para a linha 0; os endereços 4 a 7 mapeiam para a linha 1, etc.



Cache com Mapeamento Direto

- Com mesma MP de 16 blocos e um cache de 4 linhas, um outro exemplo de alocação pode ser:
 - Os locais de memória 0, 4, 8 e 12 são mapeados para a linha 0; Os endereços 1, 5, 9 e 13 mapeiam para a linha 1, etc.
- Qual é melhor?
 - Lembra de **Localidade**!
- Como podemos calcular este mapeamento?



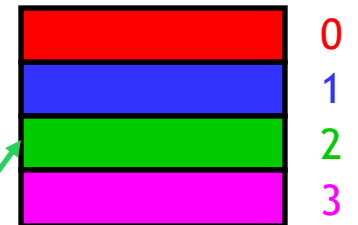
Tudo feito com a operação divisão?

- Uma maneira de descobrir para qual linha da cache um bloco de memória específico deveria ser colocado é usar o operador **mod** (resto).
- Se o cache contém 2^q linhas, então bloco i vai para
 - A linha $i \bmod 2^q$
 - Por exemplo, com o cache de quatro linhas aqui, o endereço 14 seria mapeado para a linha 2 do cache deste que
 - $14 \bmod 4 = 2$

Endereço do Bloco na MP

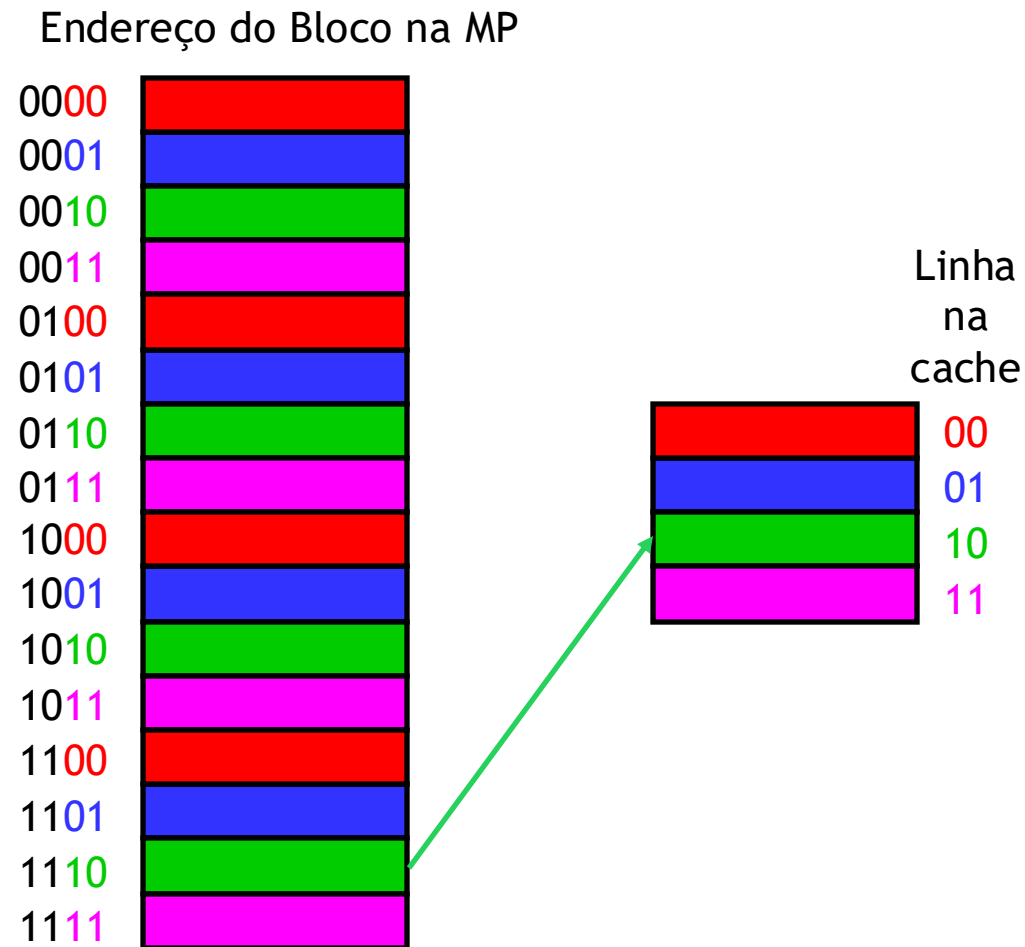


Linha na cache



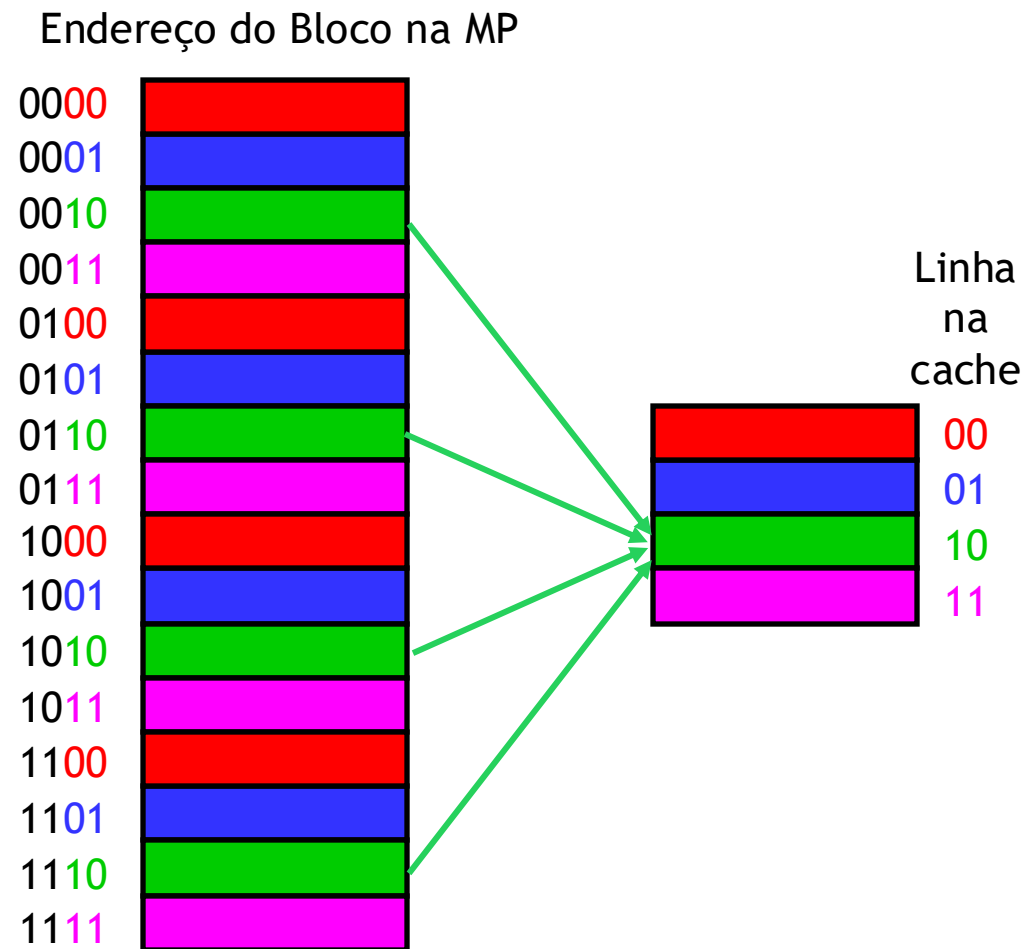
Mas divisões são operações caras ...

- Uma maneira equivalente de encontrar a localização de um endereço de memória no cache é examinar os q bits menos significativos do endereço.
- Com nosso cache de quatro linhas, inspecionaríamos os dois bits menos significativos dos endereços de bloco.
- Novamente, você pode ver que o endereço 14 (1110 em binário) mapeia para a linha 2 (10 em binário).
- Pegar últimos q bits de um valor binário é a mesma coisa de calcular **mod** 2^q do valor.
- Qual seria o custo desta operação?
 - Porque tempo importa?



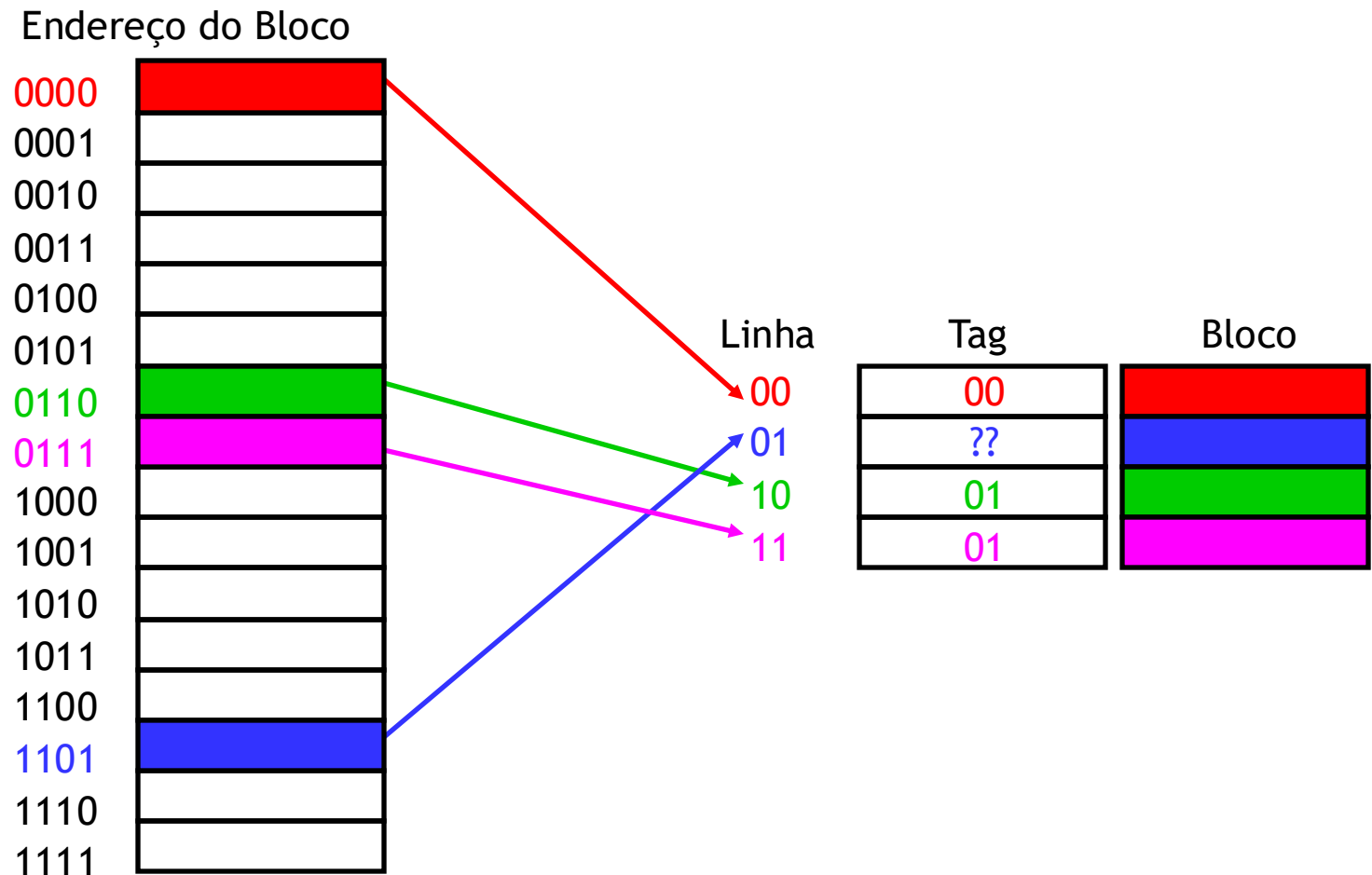
Como isso ajuda a encontrar os dados?

- A segunda pergunta era como determinar se o dado solicitado já está armazenado na cache.
- Se quisermos achar o bloco i , podemos usar o truque do **mod** para determinar em qual linha da cache poderia conter i .
- Mas lembra que outros endereços podem também mapear para a mesma linha. Como podemos distinguir entre estes blocos?
 - Por exemplo, a linha 2 da cache poderia conter um dos blocos 2, 6, 10 ou 14.
 - Precisamos saber qual bloco!



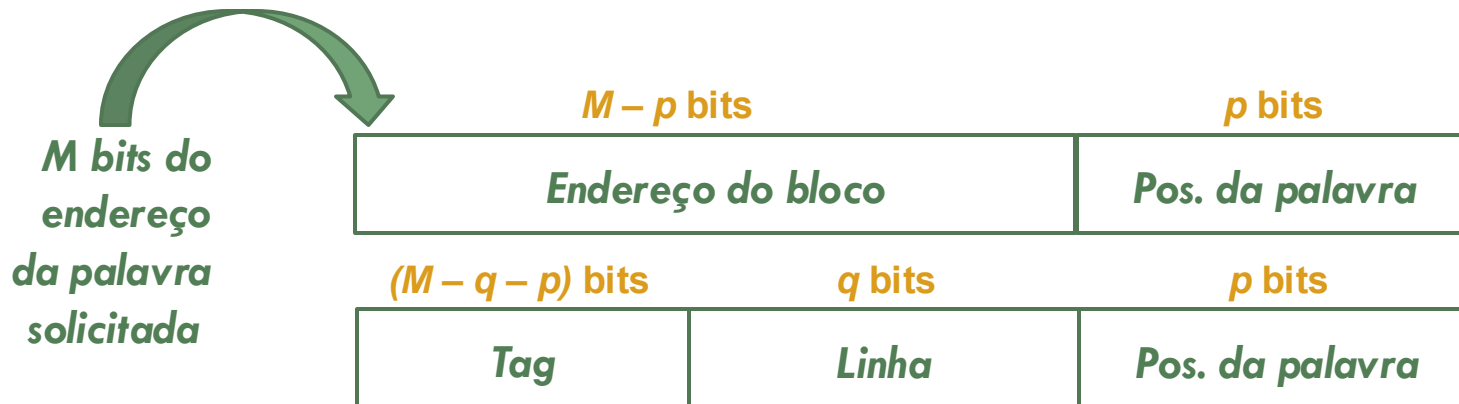
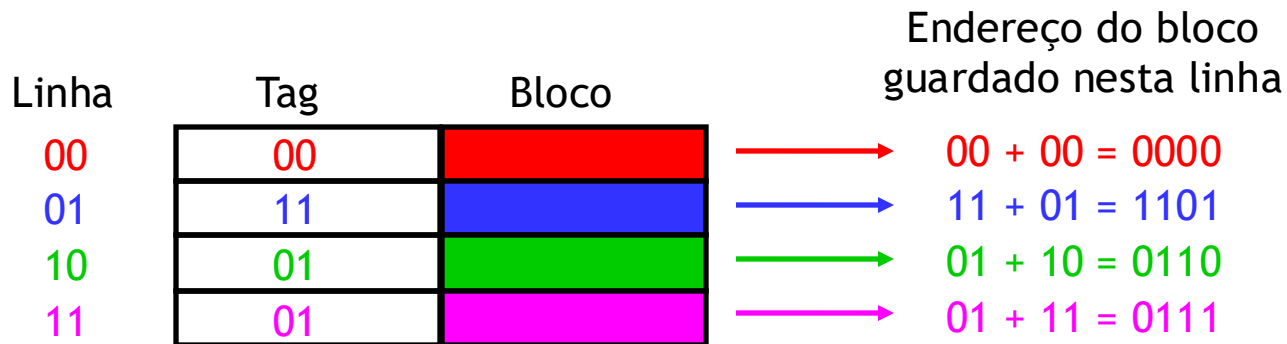
TAG x Endereço do bloco

- Usamos **Tags** na linha da cache, que fornecem os bits necessários para distinguir entre diferentes blocos que podem ser mapeados para a mesma linha do cache.



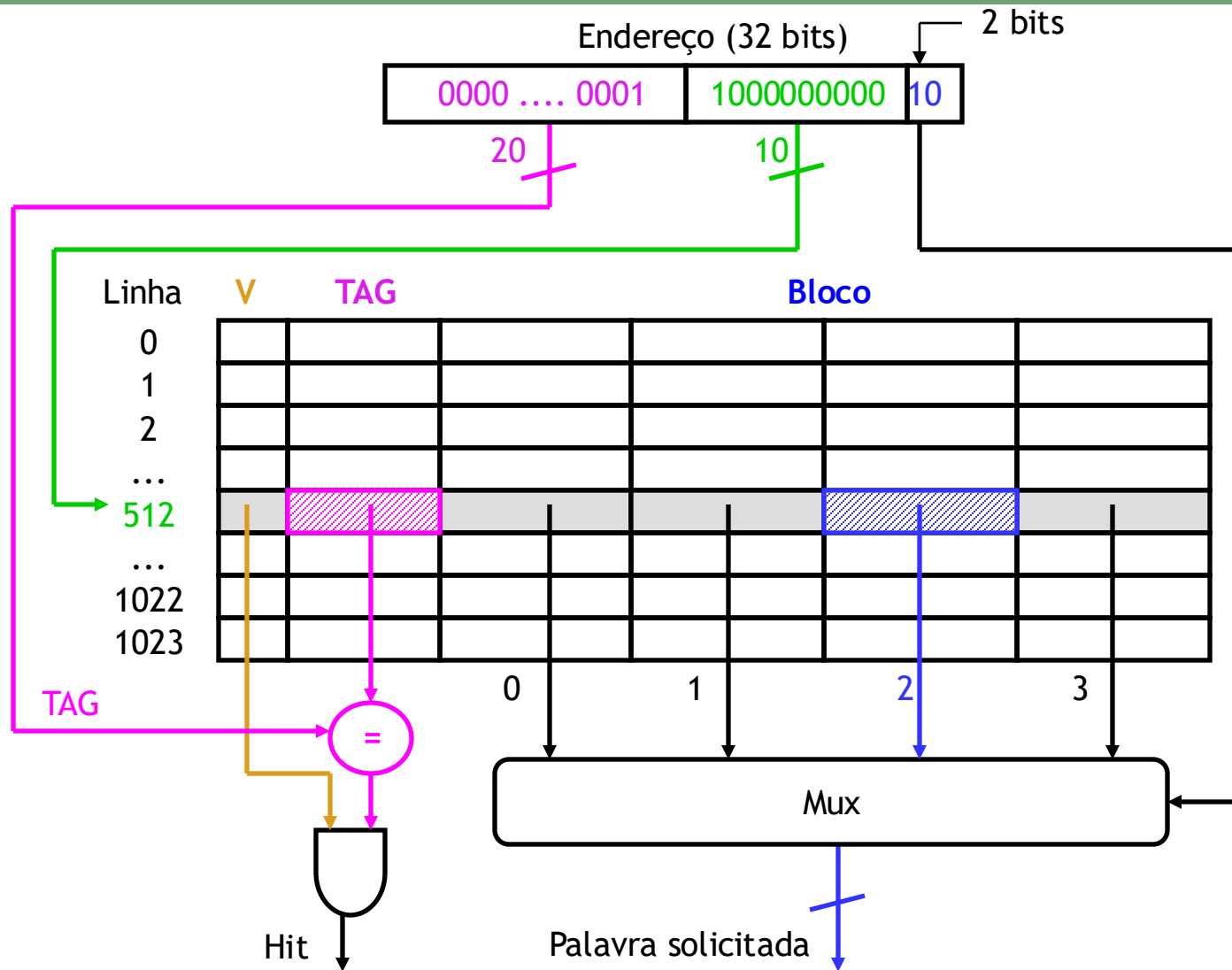
Tag x Endereço do bloco

- Em vez de armazenar todo o endereço do bloco, na cache com mapeamento direto, podemos descobrir qual bloco da MP está armazenado naquela linha, concatenando o *Tag* do bloco com o número da linha

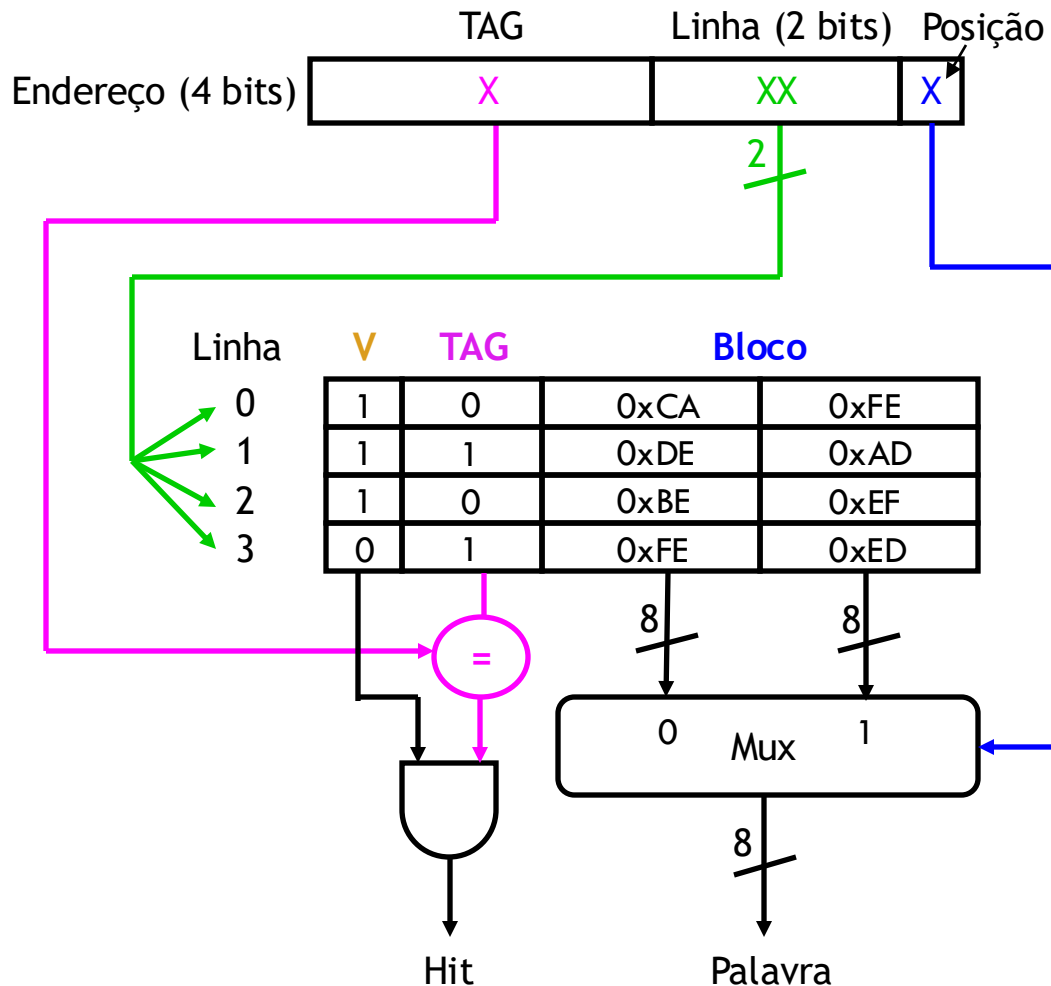


Cache com mapeamento direto

- CPU envia um endereço com que o cache determina o número do bloco que contém a palavra solicitada
- Procura na única linha que pode receber este bloco comparando o TAG do endereço com o TAG guardado



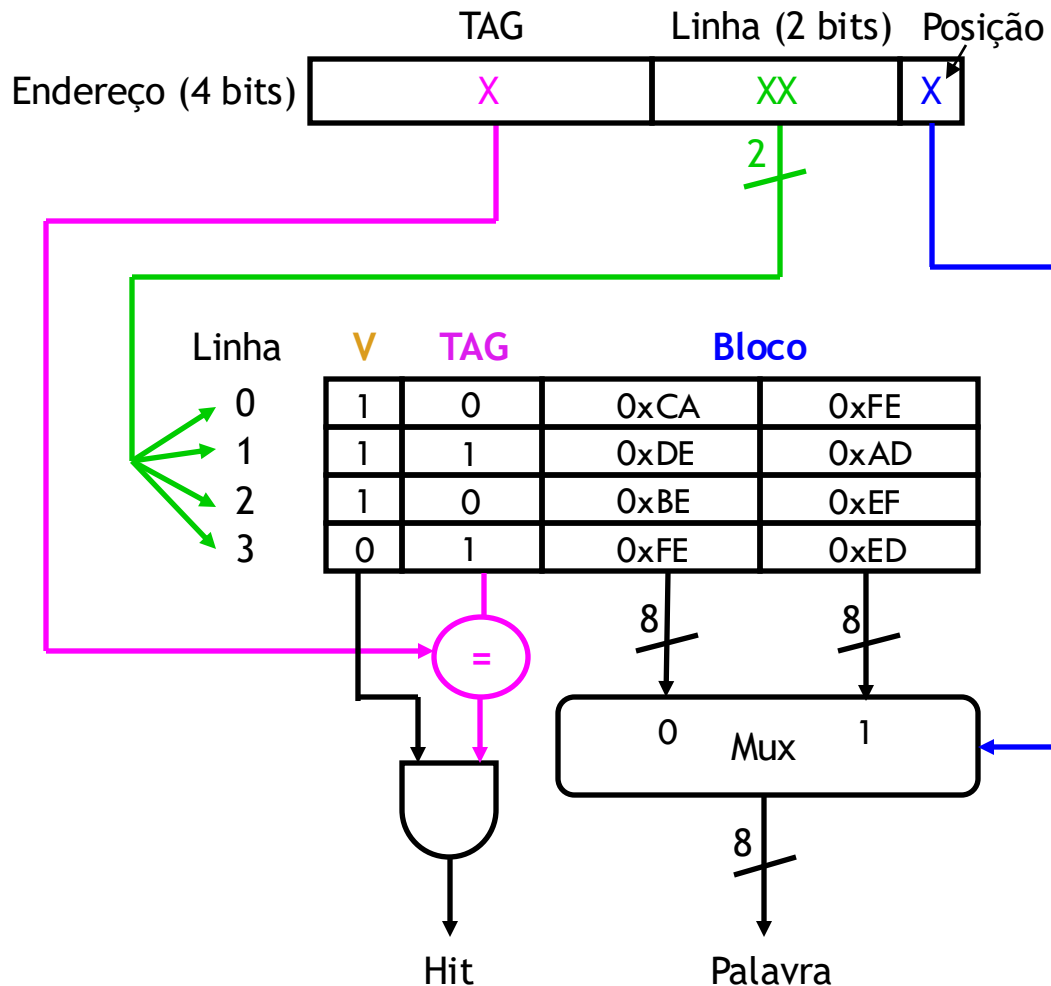
Um outro exemplo – Exercício



□ Para cada endereço em seguir, que este cache vai retornar?

- Será que uma palavra é lido da cache ou vamos ter um *cache miss*?
- 1010
- 1110
- 0001
- 1101

Um outro exemplo – Exercício



□ Para cada endereço em seguir, que este cache vai retornar?

- Será que uma palavra é lido da cache ou vamos ter um *cache miss*?
- 1010 (0xDE)
- 1110 (miss, entrada vazia)
- 0001 (0xFE)
- 1101 (miss, os Tags estão diferentes então o bloco não está na cache)

RECAP: Cache Associativa

- Neste tipo de cache, um bloco pode ser carregada em qualquer linha (disponível) da cache
- Cada linha tem três campos: um bit de validade que identifica se a linha está guardando um bloco, o número do bloco e o bloco
 - Custo do cache é relacionado seu tamanho =
 - $L \times (1 + \log_2(\text{Quantidade de blocos na MP}) + \text{Tamanho do bloco})$ bits

L Linhas	Bit de Validade	No. do Bloco	O Bloco			
0						
1						
2						
3						
...						
L - 1						

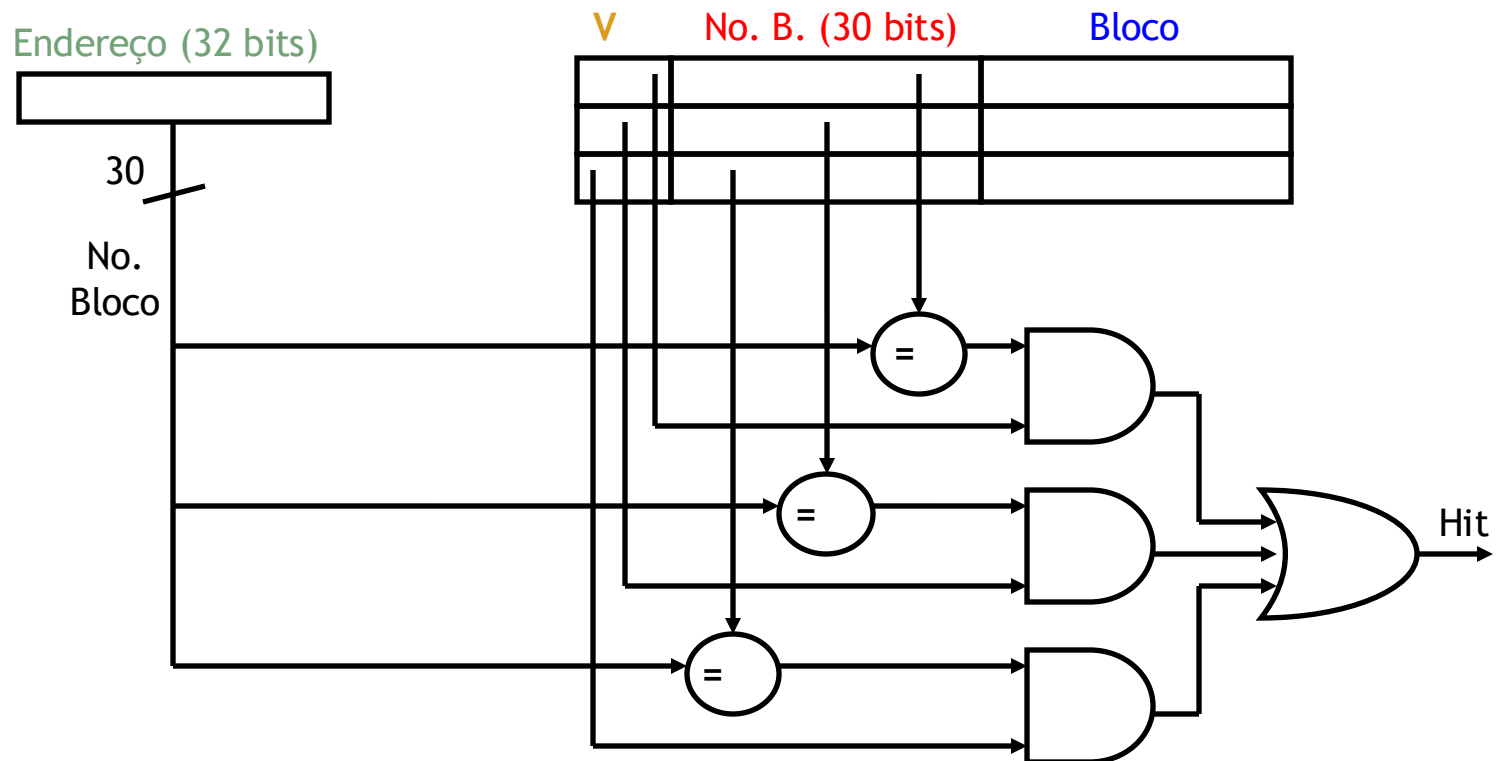
1 bit

$\log_2(\text{Tam. de MP} \div \text{Tam. do bloco})$

Tam. do bloco = $2^p \times$ Tamanho da palavra

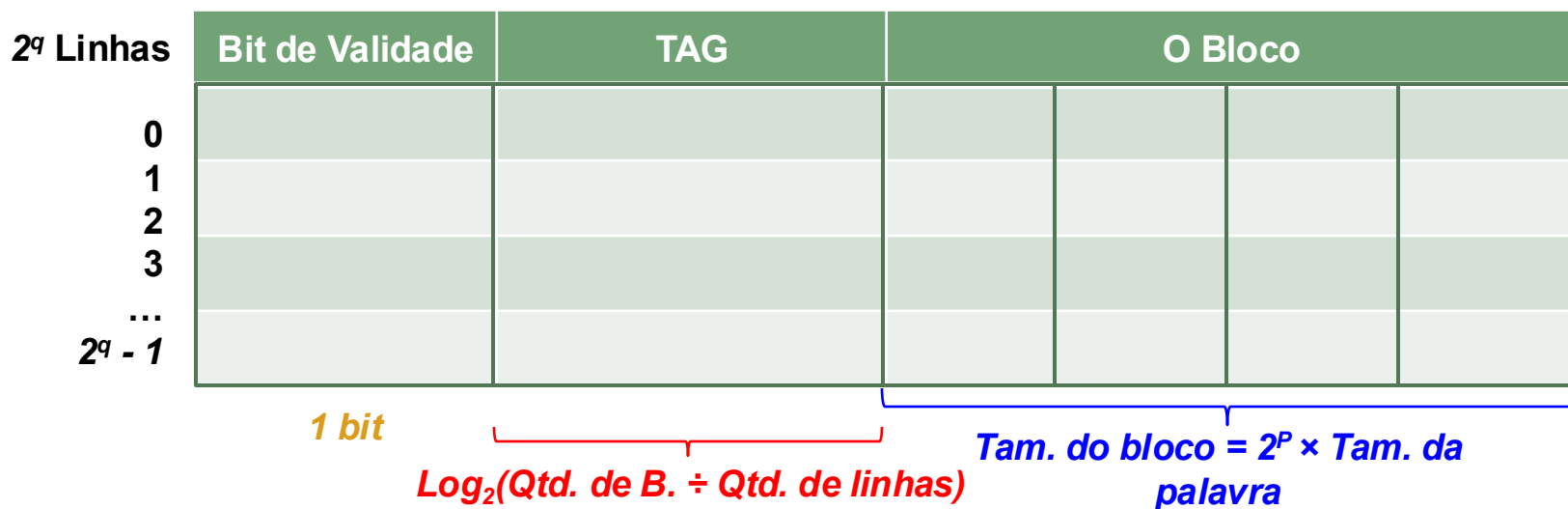
Busca em paralela da Cache Associativa

- Nesta cache um bloco pode ser carregada em qualquer linha de cache
 - Então, para achar um dado bloco, vai precisar buscar no cache inteiro
 - Para não demorar, é feita em paralelo, buscando em todas as linhas no mesmo tempo
 - Isso pode ser bastante caro, dependendo na quantidade de linhas (L) do cache



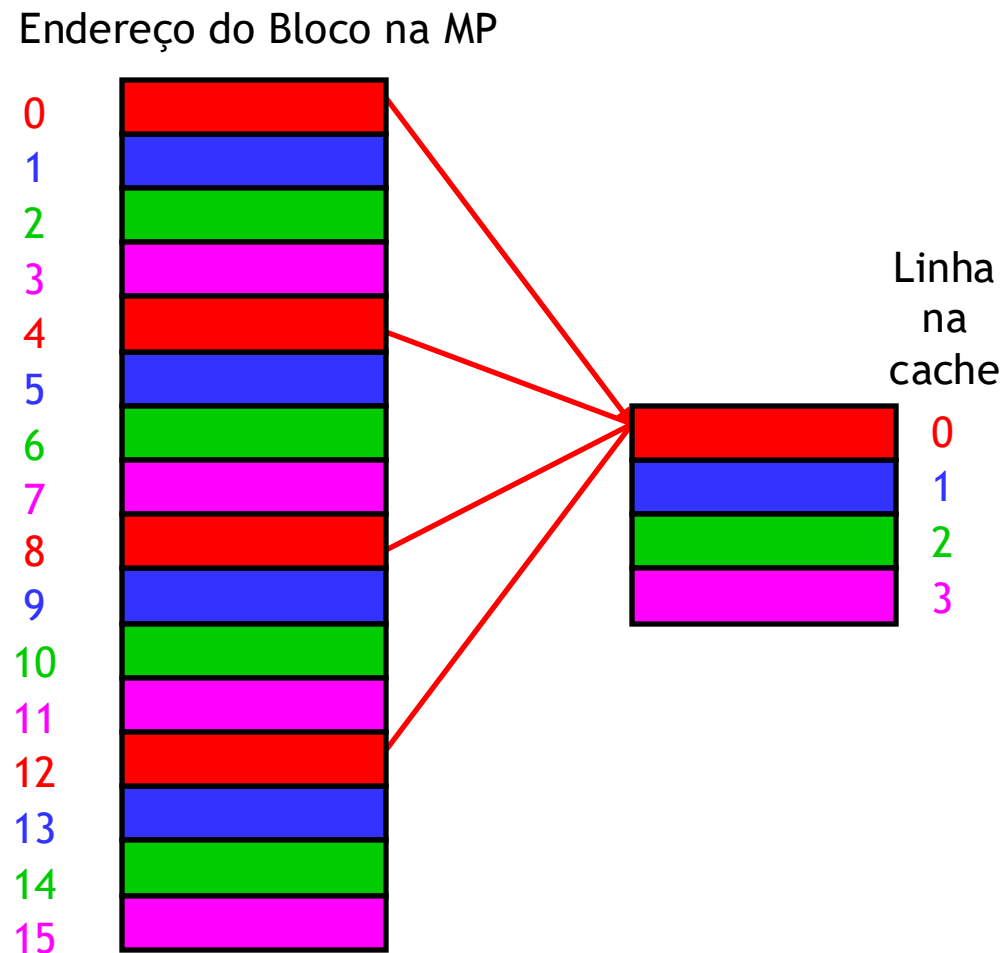
Cache com Mapeamento Direto

- Neste tipo de cache, **cada bloco** da memória principal **tem um único lugar (uma linha fixa)** na memória cache
 - Cada linha tem três campos: **o bit de validade** que identifica se a linha está guardando um bloco, **um identificador do bloco (TAG)** e **o bloco**
 - Tamanho da cache = $2^q \times (1 + \text{Log}_2(\text{Qtd de ??}) + \text{Tamanho do bloco})$



Cache com Mapeamento Direto

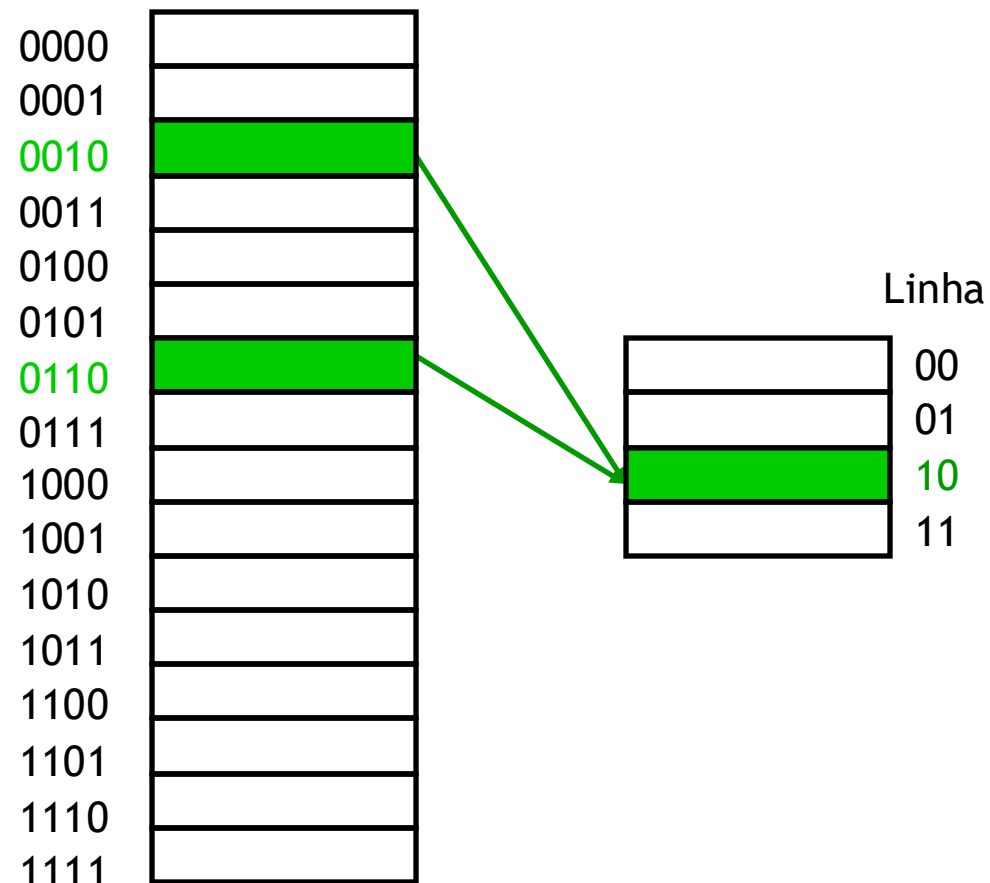
- Cada bloco tem um **único** lugar na memória cache onde pode ser guardado, mas onde?
- O mapeamento é equivalente a uma função *hash*



Cache com Mapeamento Direto

- Esta cache é mais fácil para implementar
 - As índices e deslocamentos não precisam ser calculados porque:
 - A cache tem 2^q linhas
 - Cada endereço de memória pertence a exatamente um bloco de tamanho 2^p .
- Mas, o que acontece se um programa usa endereços em blocos 2 e 6, 2 e 6, com frequência?

Endereço do Bloco



Cache com Mapeamento Direto

- Cada endereço de MP é mapeado à uma única linha da cache
- No entanto, isso não é realmente flexível.
 - Se um programa usa endereços
 - em blocos 2, depois 6, depois 2 de novo, etc., cada acesso resultará em uma falha de cache e a recarregamento da linha 2
 - Este cache tem quatro entradas, mas o mapeamento direto pode não deixar usar todas elas.
 - Isso pode resultar em mais *cache misses* do que gostaríamos.

Endereço do Bloco

0000	
0001	
0010	
0011	
0100	
0101	
0110	
0111	
1000	
1001	
1010	
1011	
1100	
1101	
1110	
1111	

Linha

	00
	01
	10
	11

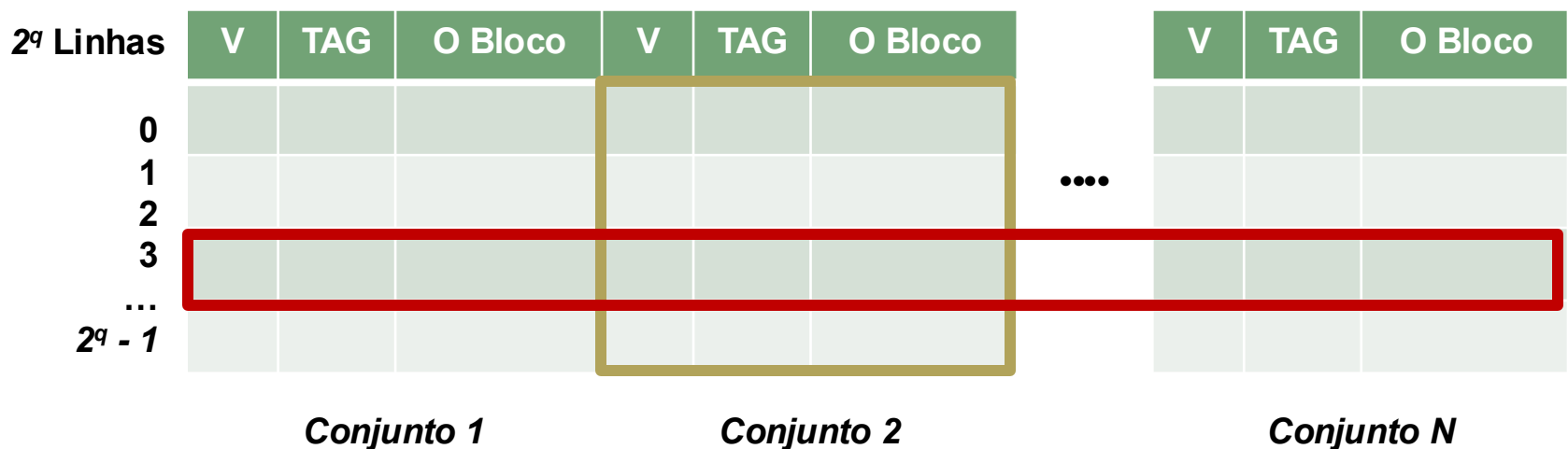
Cache Associativa por Conjunto (CAC)

- Este tipo de organização intercala as vantagens dos dois caches anteriores acabando com
 - a comparação exaustiva dos endereços de bloco nas linhas do cache totalmente associativa, e
 - o problema de conflito de endereços por uma mesma linha na cache com mapeamento direto.
- *N-way set associative cache:*

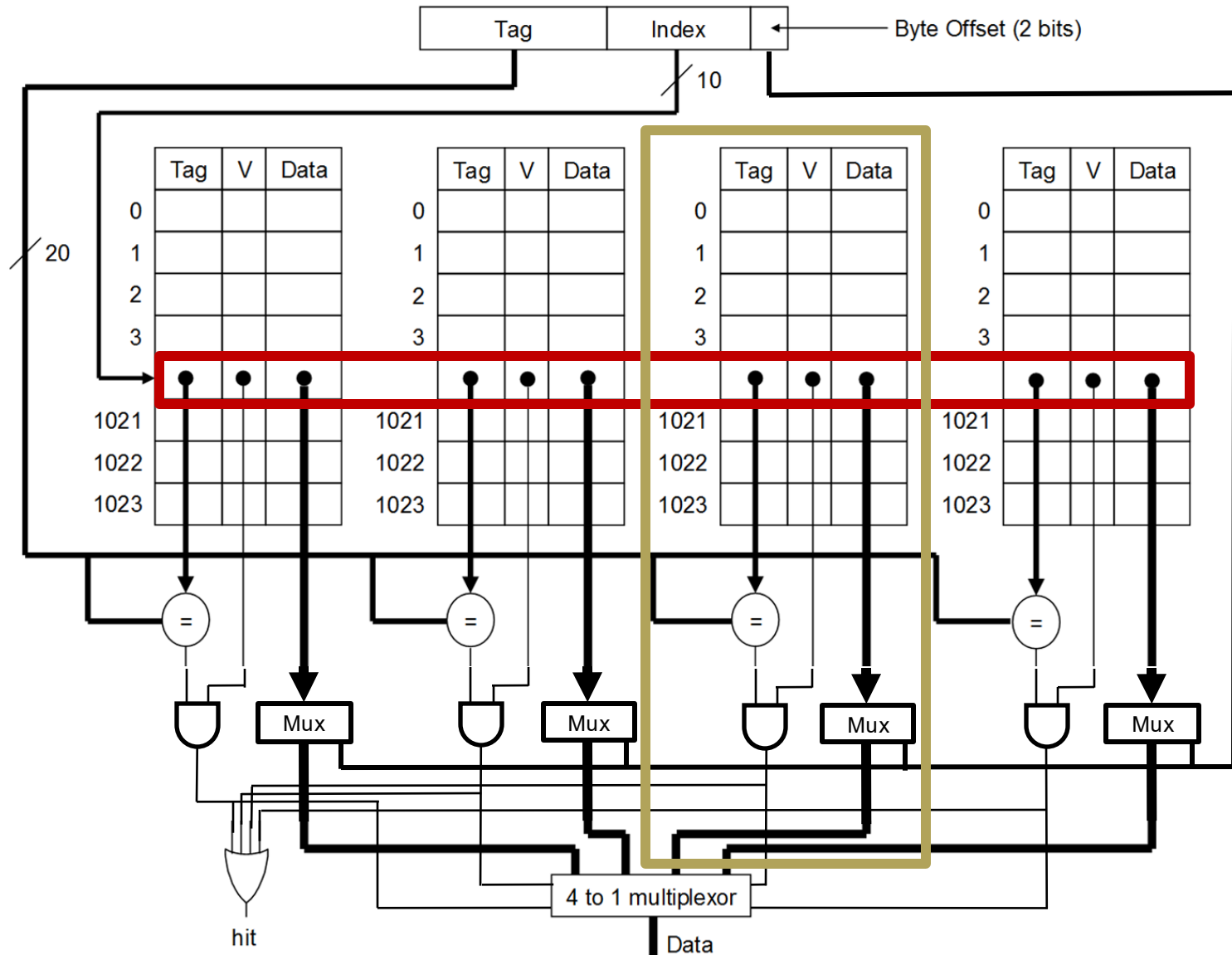


Cache Associativo por Conjunto

- A cache é composta de N **caches com mapeamento direto** coladas juntos
- Linha i de cada cache de mapeamento direto forma **um cache associativo** de tamanho N entradas
- Cada endereço de memória é mapeado para exatamente uma linha da cache (como no caso do cache com mapeamento direto), mas o bloco pode ser colocados em qualquer conjunto (ou qualquer entrada do cache associativo) daquela linha.



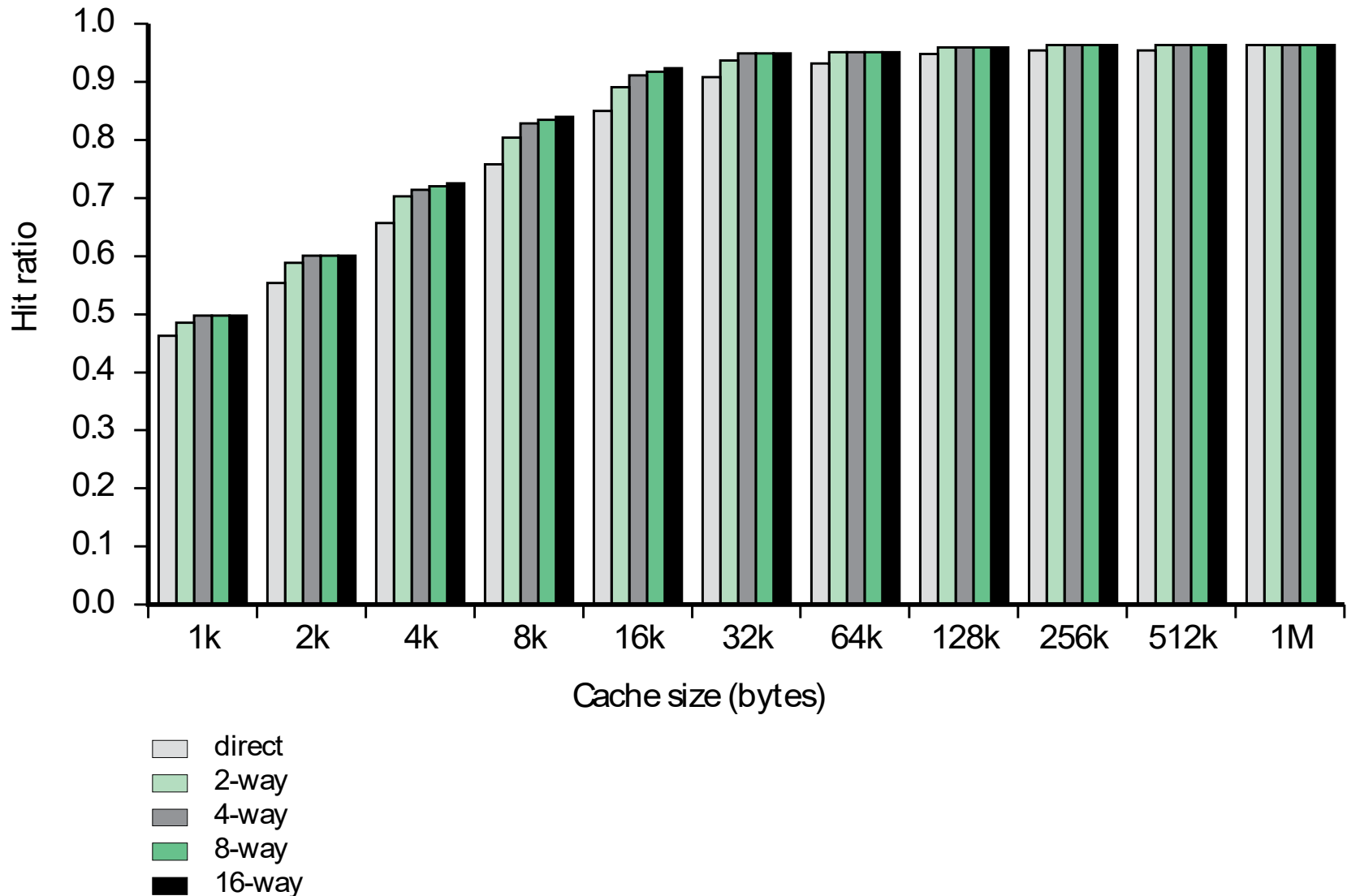
Um exemplo de CAC



Cache Associativa por Conjunto

- ❑ Tamanhos maiores de **bloco** podem tirar vantagem da **localidade espacial** carregando no cache dados não apenas de um endereço, mas também de endereços próximos
- ❑ As caches associativas mapeia cada bloco de memória a um determinado conjunto dentro da cache, mas não para uma posição específica dentro desse conjunto.
- ❑ Os tamanhos dos conjuntos podem variar de 1 (torna-se uma **cache com mapeamento direto**) a L (geralmente 2^k), a quantidade total de blocos na cache (esta seria uma **cache totalmente associativa**).
- ❑ Quantidades de conjuntos maiores e uma maior associatividade levam a menos conflitos de cache e então talvez taxas de falha mais baixas, porém também aumentam o custo de hardware.
- ❑ Na prática, caches associativas por conjunto de 2 a 16 atingem um bom equilíbrio entre baixas taxas de falha e custos.

Associatividade x Tamanho da Cache



Política de Substituição/Despejo

- No caso de uma cache com mapeamento direto, quando uma entrada/linha está ocupada, porém não com o bloco necessário, é fácil saber que este bloco teria que ser substituído.
 - O TAG do endereço e TAG da entrada serão diferentes
- No caso de caches associativas (quando está cheia) ou caches associativas por conjunto (quando a linha está cheia) e bloco não se encontra na cache, algum bloco teria que ser retirado da cache para criar espaço para o novo bloco que precisará ser trazido da memória principal.
- A questão é qual bloco deve ser escolhido para ser removido?

Política de Substituição

- O mais simples em termos de custo, seria despejar a entrada mais velha, a entrada que entrou antes as outras
 - A cache pode ser implementada como um buffer circular, sem aumentar o tamanho de cada linha
- Alternativamente, a escolha poderia ser feita numa forma randômica
 - Surpreendentemente, funciona muito bem na prática
- A melhor escolha talvez seria despejar o bloco que ficou sem uso para o maior tempo, o *Least Recently Used* (LRU)
 - Mas como implementa esta política?
 - Uma maneira necessita uma contador para cada entrada. Cada vez a cache recebe uma solicitação, todos os contadores são incrementados e se tem um hit em uma linha, o contador dela é zerado.
 - O bloco LRU é aquele que tem a maior valor no contador
 - Porém está implementação fica muito caro por necessitar um contador por linha além de ter decidir quantos bits o contador deve usar.
 - Na pratica, é usado uma forma aproximada que se chama *Pseudo LRU*
 - Dividindo a cache em grupos de entradas, o bloco mais velho do grupo LRU seria despojado.